

Evolutionary Adversarial Reprogramming of Growing Neural Cellular Automata

Aaron Bussche

David Magaram

Jason Kasdiran

Group 28

June 15, 2026

Introduction

Neural Cellular Automata (NCAs) are small, shared-weight networks that grow images from a single seed pixel and can regenerate after damage, making them a promising model for robust, self-organizing systems. Their perceived robustness has led to NCAs being deployed as a defensive component in larger architectures: Xu et al. [2] use NCA layers as plug-and-play adaptors inside Vision Transformers to make them more resistant against adversarial samples. Whether NCAs themselves can be subverted is therefore not just of theoretical interest but relevant to the security of systems that rely on them.

Randazzo et al. [3] and Cavuoti et al. [4] demonstrated targeted attacks that, for example, can make a generated lizard image lose its tail or change color. However, every published attack on NCAs relies on white-box, gradient-based methods that require full access to the model’s weights and backpropagation through the rollout. Robustness claims are so far tested only against this strong threat model. Additionally, focusing on the nature-inspired roots of NCAs, a white-box is not what a realistic adversary, for instance in a biological or bioengineering setting, would plausibly have access to.

Problem Statement

In this project we investigate whether a query-only, black-box attacker using evolutionary search can reproduce the same targeted reprogramming of a Growing NCA by changing it towards the target image, and how close does it get to the gradient-based ceiling.

The goal is to find a 16×16 state-perturbation matrix that when applied to every cell’s 16-dimensional hidden state at every timestep forces the NCA to grow into the target image rather than

to its original form (blue lizard instead of a green lizard), which is the same attack surface used by Randazzo et al. [3]. In our approach the attacker can only observe the final RGB output and optimize the L2 distance to a chosen target image using the CMA-ES evolutionary algorithm, without ever accessing model weights or gradient information.

We evaluate if our black-box attack can influence the Growing NCA in a similar way as the white-box approach by comparing the approaches by their final grown-image loss and visual output quality. If the evolutionary attacker approaches the gradient ceiling, it suggests that NCAs may be vulnerable and less robust than previously assumed. However, if it fails to closely approach that ceiling, then the NCAs may be considered safe from more realistic attack types and safe to be used in public-facing, but not open-weights, machine learning systems.

Related Work

Mordvintsev et al. [1] introduced the Growing NCA model and showed that, through a sample pool training strategy and exposure to damage during training, the learned local update rules become robust enough to regenerate damaged parts. Due to this robustness, NCAs have also been used defensively: Xu et al. [2] inserted NCA layers between vision transformer blocks to serve as an armor against adversarial attacks.

On the offensive side, Randazzo et al. [3] investigated white-box vulnerabilities of Growing NCAs and showed that, while they are more resistant to local injections, they can still be reprogrammed using global state perturbations applied to every cell at every step. Similarly, Cavuoti et al. [4] demonstrated that injecting a small number of adversarial cells into the grid can drastically alter the grown image. Both studies rely

on gradient-based optimization with full access to model weights.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES), introduced by Hansen [5], is an algorithm for non-linear, non-convex black-box optimization that maintains a population of candidates and iteratively shifts them toward high-fitness regions. While evolutionary black-box attacks are commonly used to evaluate traditional image classifiers, they have not yet been applied as an adversarial tool against cellular automata.

Methodology and Approach

Motivation

To assess the true vulnerability of NCAs under a realistic threat model, we constrain the attacker to a strict black-box setting. The attacker cannot access the internal weights of the pretrained NCA, nor can they perform automatic differentiation through the grid’s temporal evolution. Instead, the attacker can only provide an initial state (or intervening perturbation) and observe the final converged visual output. Because the fitness landscape of an NCA rolled out over hundreds of steps is highly non-convex and non-differentiable from a black-box perspective, we utilize CMA-ES. This evolutionary strategy is well-suited for our setup because it is derivative-free, robust to noisy objective functions, and scales well to our specific parameter space.

Experimental Setup

We target a pretrained Growing NCA model optimized to generate a specific 2D image from a single seed pixel (e.g., the "lizard" emoji model) introduced by Mordvintsev et al. [1]. The NCA operates on a $W \times H$ (40×40 , padding 32) grid of cells, where each cell (pixel) has a continuous 16-dimensional state vector $x \in \mathbb{R}^{16}$. Among the 16 channels, 3 encode RGB color, 1 encodes transparency (Alpha), and the remaining 12 are hidden. We can perturb the states of these cells to steer the system a certain way.

The gradient-based attack from Randazzo et al. [3], which optimizes M directly via backpropagation through the full NCA rollout using MSE loss and the Adam optimizer, with complete access to model weights, serves as the performance ceiling against which our black-box approach is measured. Following this framework, we restrict our attack to a global, linear state perturbation. At each timestep of the simulation, before the NCA’s convolutional update rules are applied, every cell’s

state vector is multiplied by a global transformation matrix $M \in \mathbb{R}^{16 \times 16}$. To maintain stability in the cellular dynamics, we restrict M to be a symmetric matrix. A symmetric 16×16 matrix contains exactly 136 unique parameters (the upper triangular elements, including the diagonal). This 136-dimensional vector space forms the search space for our CMA-ES algorithm.

To evaluate the impact of the starting distribution on the evolutionary search, we initialize the population center from three distinct points:

- **The Identity Matrix:** This represents a "zero-damage" starting point, allowing the evolutionary search to incrementally discover minimal perturbations.
- **A Random Matrix:** Initialized via a standard normal distribution to test the algorithm’s ability to converge from an arbitrary, highly destructive state.
- **The Baseline Gradient Matrix:** The final output matrix derived from the white-box gradient descent method used by Randazzo et al., establishing a benchmark for convergence speed and local minimum trapping.

For the initial standard deviation (σ_0), which dictates the initial search volume, we conduct a sweep across multiple magnitudes (e.g., $10^{-2}, 10^{-3}, 10^{-4}$). A smaller σ_0 is critical when starting from the identity matrix to prevent immediate catastrophic failure of the NCA rules.

During CMA-ES training, the NCA is rolled out for a randomly sampled number of steps in [64, 96] per generation. For evaluation, a fixed 96 steps are used. At each generation, the CMA-ES produced population of 64 candidate vectors. Each candidate is evaluated by rolling out the NCA from a batch of `batch_size`= 4 states taken from a sample pool of 1024 states and averaging the resulting loss. We apply step-size decay to avoid making disruptive jumps during search. There, σ is multiplied by `decay_factor` = 0.9 every `decay_freq` = 175 generations. The chosen hyperparameters were selected first by manual experiments, but later by 2 narrowing Optuna sweeps detailed in Appendix A.

Based on the two Optuna sweeps, our final algorithm uses the parameters displayed in table 1.

Parameter	Value
sigma0	0.0048
batch_size	4
popsize	64
decay_factor	0.9
decay_freq	175

Table 1: Hyperparameter values used for CMA-ES optimization.

Evaluation Metrics

While the Mean Squared Error (MSE, or L_2 distance) is utilized as the primary fitness function during the CMA-ES optimization process due to its computational efficiency, it is widely recognized that pixel-wise distances correlate poorly with human visual perception. A perturbation matrix may achieve a low MSE while simultaneously producing an image with unnatural artifacts or structurally disjointed features. Therefore, to rigorously evaluate and compare the outputs of the black-box evolutionary attack against the baseline gradient-based attack, we employ a suite of quantitative metrics focusing on structural, perceptual, and frequency-domain similarities.

Structural Similarity Index Measure (SSIM): Introduced by Wang et al. [7], SSIM evaluates the visual impact of luminance, contrast, and spatial structural dependencies between two images. Unlike MSE, which estimates absolute errors, SSIM considers image degradation as perceived change in structural information. We calculate the SSIM over the RGB channels with a maximum pixel value of 1.0. A higher SSIM value (closer to 1.0) indicates better structural fidelity to the target image.

Learned Perceptual Image Patch Similarity (LPIPS): To quantify semantic and perceptual similarity, we utilize the LPIPS metric [8]. By passing both the target image and the generated NCA output through a pretrained deep convolutional network (in our case, AlexNet) and comparing the internal feature activations, LPIPS effectively measures whether the generated image "looks" semantically identical to the target. Images are normalized to the $[-1, 1]$ range prior to extraction. A lower LPIPS score indicates that the adversarial image is perceptually closer to the intended target.

Target-Specific Semantic Score (TSS): While automated metrics like SSIM and LPIPS capture structural and deep feature similarities, they can occasionally overlook distinct morphological traits critical to the specific target image (e.g., the accurate formation of individual limbs

or highly localized color patterns). To bridge the gap between qualitative human perception and quantitative evaluation, we implement a Target-Specific Semantic Score (detailed in Appendix D). This metric systematically assesses five key features of the generated morphology: Granularity, Bodily Parts, Blueness, Dorsal Stripe, and Head Features. Each feature is evaluated on a discrete scale from 1 (highly degraded) to 4 (excellent target match). The final composite score $\sum TSS$ is then min-max normalized to yield a continuous value $TSS_{norm} \in [0, 1]$ according to the following formula:

$$TSS_{norm} = \frac{\sum TSS - 5}{15}$$

where a score closer to 1.0 indicates superior structural and morphological alignment with the target. The comprehensive evaluation rubric detailing the specific criteria for each score is provided in table 5.

2D Fast Fourier Transform (2D-FFT): Adversarial reprogramming of cellular automata relies on altering local convolutional updates, which frequently introduces high-frequency structural noise (e.g., checkerboard artifacts) not heavily penalized by spatial loss functions. To quantify these artifacts, we perform a 2D Fast Fourier Transform (2D-FFT) on the grayscale representations of the generated and target images. We compute the mean absolute difference of their log-magnitude spectra:

$$D_F = \frac{1}{H \times W} \sum_{u,v} \left| \log(1 + |\mathcal{F}(\text{img}_{\text{target}})_{u,v}|) - \log(1 + |\mathcal{F}(\text{img}_{\text{grown}})_{u,v}|) \right|$$

where $\mathcal{F}$ denotes the discrete 2D-FFT.

Furthermore, because our targets involve localized morphological changes (such as the removal of a lizard’s tail or head), a global Fourier transform may average out highly localized frequency discrepancies. To address this, we implement a partitioned spectral difference metric: the images are divided into a 2×2 grid (four quadrants), and the spectral difference is calculated independently for each block and then averaged. This allows us to precisely capture localized artifacting around the explicitly altered regions of the target shapes.

Different Fitness Functions

At each generation, every candidate perturbation matrix is evaluated by rolling out the NCA for a batch of states and computing a scalar distance

between the generated output and the target image. CMA-ES then uses this to update its search distribution.

Five different fitness functions were evaluated in total. All were evaluated on the same color change task across 500 generations using identical parameters found in table 1. Fitness functions were selected based on their relevance to this task, with MSE serving as the baseline, and evaluated using the evaluation metrics mentioned earlier.

Mean Squared Error (MSE): MSE measures the average of the squared differences between the actual and predicted values. It is a pixel-wise fitness function that is applied to all 4 channels, where x is the generated image and t is the target image. It squares the errors, giving more weight to larger errors, and is used as a baseline in this project.

$$\mathcal{L}_{\text{MSE}}(x) = \frac{1}{HWC} \sum_{h,w,c} (x_{hwc} - t_{hwc})^2$$

Huber: Huber loss [9] is a robust fitness function which combines MSE and MAE. It uses MSE for small errors and switches to MAE for larger errors when δ is crossed. We use a small $\delta = 0.1$ threshold to use MSE for pixel errors smaller than 10% of the full range and MAE for larger errors.

$$\ell_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & |r| > \delta \end{cases}$$

Weighted RGBA: This is a variant of the MSE function that gives a higher weight to the specified channels: $\mathcal{L} = \frac{1}{HWC} \sum_{h,w,c} w_c (x_{hwc} - t_{hwc})^2$. Weights were manually set to $w = 2$ for RGB and $w = 1$ for A.

Foreground-masked MSE: This fitness function is a spatially weighted variant of MSE that scales each pixel’s contribution by its position in the image. It works similarly to weighted RGBA, however, all channels are weighted equally, pixels inside the body contribute fully, but other pixels contribute less, with background pixels contributing only 20%.

$$\mathcal{L}_{\text{fg}}(x) = \frac{1}{HWC} \sum_{h,w,c} (0.2 + 0.8 t_{hw,\alpha}) \cdot (x_{hwc} - t_{hwc})^2$$

SSIM: As detailed in the Evaluation Metrics section, we also explore SSIM [7] as an alternative fitness function to explicitly optimize for structural and perceptual similarity.

$$\mathcal{L}_{\text{SSIM}}(x) = 1 - \frac{1}{|W|} \sum_{w \in W} \frac{(2\mu_x \mu_t + c_1)(2\sigma_{xt} + c_2)}{(\mu_x^2 + \mu_t^2 + c_1)(\sigma_x^2 + \sigma_t^2 + c_2)}$$

Results

Experiment 1: Comparing our CMA-ES approach with their gradient-based approach

1.1: Comparison after 30 Minutes runtime

Averaged over four independent runs, the gradient-based model reaches a slightly lower final loss in the same time, but the black-box CMA-ES attack stays close (figure 1). A paired comparison of the grown-image loss across the four runs shows the gradient baseline lower in every run (mean 0.00377 ± 0.00017 versus 0.00414 ± 0.00015), although the gap is small and only marginally significant ($p \approx 0.07$, paired t -test, $n = 4$). Both methods lie roughly five times below the empty-seed baseline loss of 0.0218, confirming that both clearly succeed at the attack.

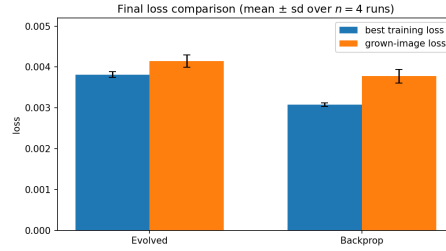


Figure 1: The best loss achieved during training and the grown-image loss of the final model, for both our implementation and the gradient-based implementation after 30 minutes of training time. Bars show the mean over $n = 4$ independent runs and error bars the across-run standard deviation.

To compare the optimization efficiency of the black-box CMA-ES attack against the gradient-based baseline, we tracked the convergence of the $\log_{10}$ objective loss across three different independent variables, namely wall-clock time, algorithmic iterations, and cumulative NCA state-steps. Given the fundamentally different mechanics of population-based search versus gradient descent, relying on a single metric does not provide the full picture. Wall-clock time serves as a practical benchmark for real-world adversarial feasibility; by this measure, the evolutionary attack is highly competitive. Figure 2 shows that the gradient-based baseline completes significantly more update steps and converges to a marginally lower final loss than the CMA-ES attack. While the loss per iteration (figure 3a) appears superficially comparable, this metric masks the differing computational payloads. A single CMA-ES iteration evaluates an entire population of candidates,

whereas the gradient-based iteration evaluates a single mini-batch. The loss per cumulative NCA state-steps (figure 3b) isolates forward-pass sample efficiency, highlighting that the CMA-ES attack requires roughly three times as many cumulative simulation steps to reach a comparable loss as the gradient-based baseline. Thus, when analyzing the loss per iteration or per cumulative NCA state-step, the inherent sample inefficiency of the CMA-ES attack becomes apparent.

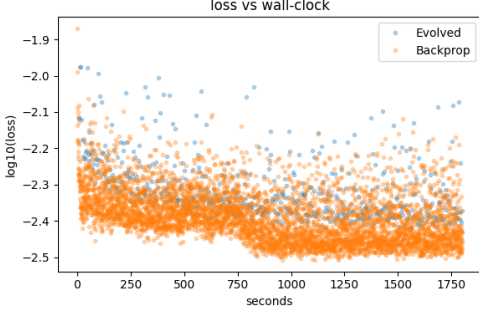
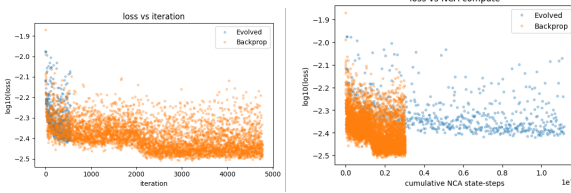


Figure 2: Optimization trajectory measured by wall-clock time (approximately 30 minutes).



(a) Optimization trajectory measured by training iterations. (b) Optimization efficiency measured by cumulative NCA state-steps.

Additionally, we evaluate the visual results of both attacks under two different conditions. Figure 4 shows the results when the perturbation matrix is applied from a single seed pixel to grow the target shape. Figure 5 shows how the attacks behave when applied to a fully grown original NCA lizard to mutate its existing structure. Figure 6 further details the step by step growth process of both attacks over 120 simulation steps.



Figure 4: Visual comparison of attacks grown from a single seed pixel. Left: The target image for our attack. Middle: Lizard grown from pixel with CMA-ES attack (ours). Right: Lizard grown from pixel with gradient-based attack (theirs).



Figure 5: Visual comparison of attacks applied to an already grown structure. Left: fully grown lizard, original NCA lizard. Middle: CMA-ES attack on fully grown lizard. Right: gradient-based attack on fully grown lizard

Visually, both the gradient attack and our CMA-ES attack produce noisier results than the original trained NCA and the target image. This noise is obvious up close but blends in from a distance. The CMA-ES lizard has sharper and more ragged edges. It retains some original green colors around the edges of the limbs and tail, while a darker blue shines through more overall. The gradient attacked lizard has better color accuracy and superior fine details, like the eye position and the white line on the back.



(a) Gradient-based attack growth progress.



(b) Evolutionary attack growth progress.

Figure 6: Attacked Lizard during growing steps 0, 20, 40, 60, 80, 100, 120.

That said, it is not as easy as to say that the gradient-based attack produces a blurred version of the target image. Figure 7 shows that a blurred version of the target image is of brighter colour, with cleaner fine details (such as the eyes and tail).



Figure 7: Reference image showing the target transformed down and then up four times with linear, none, and cubic interpolation algorithms in GIMP.

Furthermore, we analyze the mathematical differences in the final perturbation matrices.

The changes from identity (not shown in Figure 8) are hard to see visually in the CMA-ES version, as they are comparatively tiny. However, in figure 8 we see that the gradient baseline has made much bigger changes to the starter perturbation matrix, as also validated by comparing the

Euclidean distance between the perturbation matrices and the identity matrix. This is despite leading to comparable (if worse) results.

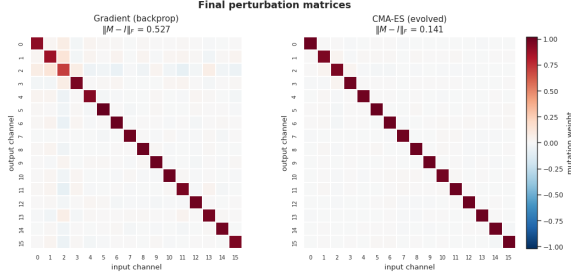


Figure 8: Visualisations of the final perturbation matrices for the gradient baseline and our CMA-ES approach, both trained 30 minutes.

Finally, Table 2 presents the quantitative evaluation of both attack methods, averaged over the four runs. Across the structural, perceptual, and frequency-domain metrics (SSIM, LPIPS, and 2D-FFT) the two methods are statistically indistinguishable: none of the paired differences are significant ($p = 0.21, 0.99$, and 0.82 respectively, paired t -test, $n = 4$), and the LPIPS and 2D-FFT means are essentially identical. The Target-Specific Score (TSS), which we scored on a single representative run, favours the gradient baseline (0.533 versus 0.467), consistent with our visual observation that the gradient attack better reconstructs semantic features such as the eyes and the dorsal stripe. Overall, the only consistent quantitative gap is the slightly lower grown-image loss of the gradient baseline; on every perceptual metric the black-box attack closely follows it.

Method	Grown loss ↓	SSIM ↑	LPIPS ↓	2D-FFT ↓	TSS ↑
CMA-ES	0.00414±0.00015	0.933±0.001	0.077±0.009	0.438±0.006	0.467
Gradient	0.00377±0.00017	0.937±0.005	0.077±0.004	0.440±0.010	0.533

Table 2: Quantitative comparison of the CMA-ES approach and the gradient baseline, reported as mean $\pm$ standard deviation over $n = 4$ independent runs (TSS scored on a single representative run). Higher SSIM and TSS, and lower grown-image loss, LPIPS, and 2D-FFT, indicate better fidelity to the target image.

Experiment 1.2: Environmental Impact

To contextualize the computational cost and environmental footprint of different methods, we tracked carbon emissions and energy consumption during the optimization process. For a fair comparison of convergence efficiency, each method was run until it reached a target $\log_{10}$ loss of -2.5.

As detailed in Table 3 and illustrated in Figure 9, the gradient approach is substantially more efficient. It successfully reached the target loss

in roughly half the time and consumed approximately half the overall energy compared to the evolutionary approach.

Method	Emissions	Duration (s)	Energy (kWh)
CMA-ES	0.015	1994.738	0.043
Gradient	0.008	1024.074	0.022

Table 3: Environmental impact and computational cost of the evolutionary strategy and the gradient baseline. The total carbon emissions are presented in kgCO_2e .

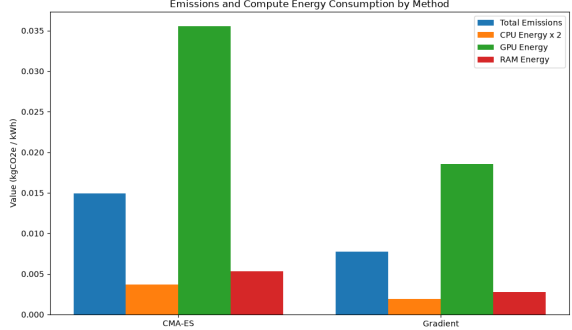


Figure 9: Comparison of total carbon emissions and energy consumption breakdown (CPU, GPU, and RAM) across the evaluated methods.

Experiment 2: How long does it takes to recover from an attack: Perturbation On/Off

Following the same steps as Randazzo et al. [3] did in their experiments, we also explore how long it takes to recover from an attack by setting perturbation on and off. The visualization of the training run ran for 2500 steps while toggling perturbation off during steps 500-1000. A model trained for 500 steps with the MSE loss is used.



Figure 10: Images from the visualization of the training run at steps 100, 500, 515, 530, and 2500, with perturbation off from 500 to 1000, trained for 500 steps.

Figure 10 displays the results at specific steps during the visualization of the training run. Earlier steps are not displayed as Figure 6b already depicts them. What is important to note is that by step 60 the lizard is fully initialized, with step 500 here serving purely as a baseline. Turning perturbation off, within 15 steps the lizard loses most of the blue color, and by step 530 the blue is entirely gone. By step 625 the image grows sharper,

and by step 925 we can notice its individual toes, making it almost indistinguishable from the original. Turning the perturbation back on, the image behaves similarly as it did in the beginning. However, at step 1800 it begins mutating, creating a new body by step 2500.

In the end, it only takes the model several steps to regain its original color and several hundred steps to return to its original state, demonstrating its impressive regenerating capabilities. While it began mutating, it is not uncommon. We attribute this to the fact that the model has only been trained for 500 steps.

Experiment 3: Change Lizard to Butterfly

In order to explore the capabilities of the NCA model and the CMA-ES framework, we tried to change the original lizard into a butterfly. For this, we only needed to change the target image.

Figure 11 displays 5 images from the visualization of the training run at 5 different steps, along with the target image for comparison. The first image at step 120 was selected because it had the most butterfly-like shape. The overall color distribution also approximates the target image distribution, however, the details and patterns are not present. At 500 steps, the butterfly shape is somewhat lost. After 500 steps, perturbation is turned off and the NCA starts regenerating. But because of the strange shape and pixel distribution, 2 lizards are regenerated. After 1000 steps, perturbation is turned back on. Again, after 120 steps the shape is the most butterfly-like, however, this time there are 2 lizards so we end up with 2 overlapping butterflies. At the end at 2500 steps the NCA again loses its shape in a similar fashion as at 500 steps.

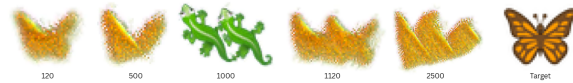


Figure 11: Images from the visualization of the training run at steps 120, 500, 1000, 1120, and 2500, with perturbation off from 500 to 1000, trained for 600 steps. Target image included.

While the target image was not reached, it can be concluded that the task succeeded to some degree as CMA-ES managed to approximate the butterfly, both its shape and its orange color. This also demonstrates the CMA-ES black-box capabilities and the NCA’s impressive regenerative powers. While the results are not exactly perfect, had the model been trained for longer we expect the outcome to be slightly better.

Experiment 4: Different starter perturbation matrices

Instead of starting from identity as attack matrix, we also tried starting from white noise & the 8000 step gradient-optimized matrix. We used the same hyperparameter values we found using optuna to train a 500-generation attack starting from the identity matrix.

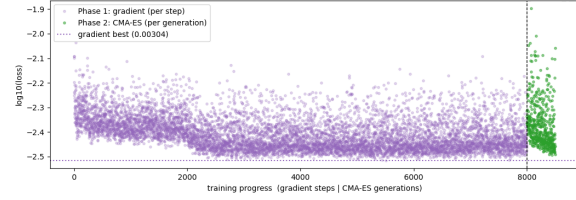


Figure 12: The $\log_{10}$ loss of the gradient approach (theirs) in purple trained for 8000 steps, with the CMA-ES approach (ours) taking its final perturbation matrix as starter.

There is initially a significant jump upwards in loss after the switch to CMA-ES, likely caused by σ_0 being too high and too big steps being taken away from the already decent local optimum. The loss does recover though to a similar level after 500 generations.

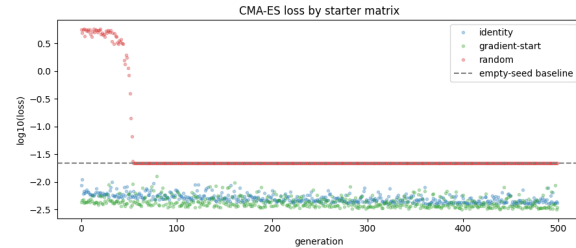


Figure 13: The $\log_{10}$ loss of 3 training runs against the baseline: starting with the identity matrix (default), the already gradient-approach optimized matrix, and one filled with gaussian noise.

While the model trained starting from the identity matrix and the model trained from the gradient-result-matrix continued improving, the white-noise matrix start only managed to not be worse than producing a white canvas and got stuck there. The baseline here is the loss for the starter image of white canvas with a single black pixel in the middle, so very similar to a fully white canvas.

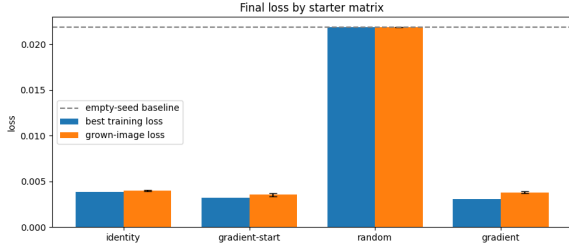


Figure 14: The best loss during training and the average final loss after growing a full lizard for each of the starter matrices and the gradient approach (theirs) for comparison.

In averaged loss after growing a full lizard from a pixel, the random matrix start sits near the empty baseline. Identity comes third place at 0.0040, gradient-only (theirs) is second at 0.0038 (± 0.000086) and the best result comes from further training the gradient output with CMA-ES at 0.0036 (± 0.000177). Keep in mind that the gradient-trained and CMA-ES trained results here are not directly comparable, as they are not matched in computation time or another way.



Figure 15: Target (nr.1) and gradient-trained (nr. 4) lizards among CMA-ES trained ones. The third one started training on the gradient-trained matrix.

The output from the white-noise trained matrix is just white, while the other ones clearly show a lizard. The Lizard trained from the gradient-trained baseline matrix seems to have the best colour accuracy, while the gradient-trained one has the most detailed white spine.

Experiment 4 Addition: Training a gradient-approach attack starting with a random perturbation matrix

As we were interested in how the gradient-based approach would handle a random starter permutation matrix in comparison, we additionally also ran a white noise start with it. Instead of producing white outputs, it looked like this after 8000 training steps:

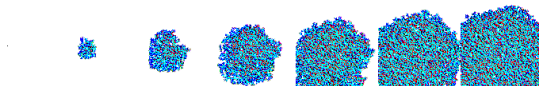


Figure 16: Steps 0, 20, 40, 60, 80, 100, 120 of a gradient-based model (theirs) trained for 8000 steps starting with a gaussian permutation matrix.

With a grown-image loss of 4.550461 (± 0.201547) it never even came close to the baseline of one starter pixel on a white background as shown in Figure 14 and has a much higher loss than the rest. More figures can be found in Appendix C.

Experiment 5: Different Fitness Functions

Table 4 displays the evaluation metrics for the explored fitness functions.

Fitness Functions	SSIM $\uparrow$	LPIPS $\downarrow$	2D-FFT $\downarrow$	TSS $\uparrow$
MSE	0.925	0.096	0.541	0.47
Huber	0.935	0.097	0.495	0.47
Weighted RGBA	0.934	0.103	0.544	0.47
Foreground-masked MSE	0.927	0.117	0.576	0.27
SSIM $\star$	0.950	0.197	0.362	0.40

Table 4: Fitness functions with their evaluation metrics. (lower) or (higher) determine if a smaller or larger scores are better. $\star$ means that the lizard did not turn blue.

As can be seen from the table, all losses perform similarly well. However, out of all the losses, Huber performs the best overall, achieving the highest SSIM of 0.9349, the lowest FFT of 0.4949, and the second lowest LPIPS of 0.0972.

While quantitative metrics are a good indicator of performance, they do not fully reflect the actual results. In order to do that, we need to also inspect the produced final images displayed in Figure 17, which shows the mean grown image averaged over 8 rollouts using the final perturbation matrix for each of the losses, along with the target image. All losses, except the SSIM, can successfully recolor the lizard, however, there are visible differences. Huber produces the sharpest looking result consistent with its performance across all metrics, but the limbs and the head have green residue and the edges of the body are much darker compared to others. MSE produces a similar result with slightly better colors. Although it is less sharp and lags behind in some metrics, the results look to be on par with Huber. The weighted RGBA and the Foreground-masked MSE both produce noticeably blurry results. Comparing the two, weighted RGBA better handles color and does not produce pink eyes. Lastly, although the SSIM has technically the highest SSIM and lowest FFT, it fails the task of turning the lizard blue. This is most likely due to the fact that the green lizard to a large degree already satisfies its measures of luminance, contrast, and structure. This is an example of a phenomenon called Goodhart’s law [10] which states that when a measure becomes a target, it ceases to be a good measure.

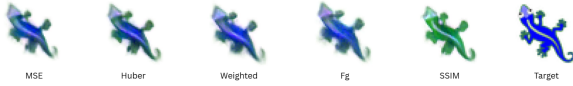


Figure 17: Mean grown image averaged over 8 roll-outs using the final perturbation matrix for each of the 5 losses, along with the target image.

Overall, Huber provides the best results closest to the target, both quantitatively and visually. The results show that the selection of loss function affects the quality of the output and the attack in general.

Discussion

Compared to the original backpropagation version, our NCA-ES tuned adversarial attack is less effective in the same time. That said, the evolutionary version is in our opinion surprisingly not that far behind and produces results which are roughly on the same qualitative level as the ones presented by [3].

Comparing these two methods highlights a tradeoff between threat models and overall resource costs. Because CMA-ES evaluates entire populations instead of using gradients, it requires roughly three times more forward simulation steps. Despite this computational penalty, the evolutionary attack remains competitive in practical wall clock time. However, when we measure the environmental footprint required to reach a specific target loss of -2.5 (on a $\log_{10}$ scale), the differences become obvious. The gradient approach is significantly more efficient. It reaches the target in half the time, uses half the total energy, and produces half the carbon emissions compared to the CMA-ES method. While the gradient baseline is greener and faster, it is less stable. We observed that the gradient attack produces a higher standard deviation in its final loss. Gradient descent quickly exploits local information but often gets trapped in local minima based on its starting conditions. CMA-ES avoids this issue by continuously sampling a broader search space. This guides the model toward more stable solutions and results in less variance across different runs.

Now CMA-ES and the original version are both relatively successfully optimizing the same loss function, and getting to similar results with differing amounts of effort. But there are still differences in the results they produce. The end result of the gradient-optimized attack is an image with more details, but it overemphasizes the blue and is somewhat blurry. The CMA-ES trained adversarial attack leads to a more jagged image that perhaps overemphasizes the green of the edges and is

more jagged and grainy, but not washed out.

Crucially, both optimizers optimize in broadly the same direction and our version can further improve on top of the final results by the original approach. Seeing that CMA-ES does not lead away from the optimum but further guides toward it validates our approach. The results would likely be even better if we started the run (see figure 12) with a lower *sigma0* instead of the priorly found optimal values for full training, so that it hadn't initially taken disproportionately big steps out of the local optimum. It is interesting though that the results of both optimizers do clearly have distinct characteristic looks given the same training objective, meaning they do not optimize in precisely the same direction.

Our approach does not manage to recover from being initialized from a fully gaussian starter matrix, but instead produces a completely white image. These results are very similar to when we first started experimenting with our approach and, underestimating the power of the permutation matrix being multiplied once per growth step for a total of 64-96 times, set *sigma0* too high. The big implications of small changes likely lock the model in a very unfavorable local optima that it cannot manage to climb out of, as every single step from it makes the results even worse. Interestingly, the gradient descent approach actually does much worse than ours when starting with a gaussian perturbation matrix in terms of loss, growing something like blue TV static. A likely explanation is that, since M is reapplied at every one of the 64-96 growth steps, backpropagating through such a deeply iterated, chaotic rollout yields ill-conditioned gradients that overshoot into a worse attractor, whereas the derivative-free CMA-ES cannot be pushed below its starting quality and instead settles on the safe near-white canvas.

The standard identity matrix starter is a forgiving start, as it preserves all features of the lizard, with only the colour needing changing, so a step-by-step improvement is possible. When analyzing the final perturbation matrices, we observe that our approach stays very close to the identity matrix while theirs strays far, which is unexpected for us. This indicates that our approach reaches roughly the same results with a much smaller perturbation. Since the magnitude of perturbation needed to subvert a model reflects how (non-)robust it is, a near-identity attack arguably strengthens our claim that NCAs are less robust than assumed. A perturbation close to identity also leaves most of the NCA's learned local update rules intact and only gently steers the dynamics,

which may relate to the lower variance in our final loss (Experiment 1).

Experiment 3, the lizard to butterfly experiment, demonstrates that the black-box attack can be used for other types of attack, just by simply swapping the target. CMA-ES can approximate the butterfly’s shape and color distribution to some degree and grow back to its original state when perturbation is turned off. While it mutates one perturbation is applied again, it shows that even if the NCA’s underlying dynamics are of a lizard we can steer the output towards a different target without permanently changing what the model has learned.

Similarly, in Experiment 2, where we turn perturbation on and off, the NCA model displays impressive regenerative capabilities. Once the perturbation is disabled, the blue coloring disappears within 15 steps and is entirely gone within 30 steps. Throughout the next steps the lizard then recovers its structural details and is almost indistinguishable from the original by step 925. While NCAs are used as defensive layers by Xu et al. [2], it is important to note that they can be attacked using a black-box attacker, however, the effects are not permanent as once the perturbation is disabled, they regenerate back quickly.

The fitness function comparison in Experiment 5 shows that the selection of loss function affects the quality of the output and the attack in general. Huber achieves the highest SSIM of 0.9349 and lowest 2D-FFT of 0.4949, thanks to its blend of MSE and MAE. Foreground-masked MSE and Weighted RGBA compare poorly as they introduce noticeable blur, suggesting that channel based losses are not ideal. The SSIM loss achieved an even better SSIM and 2D-FFT scores than Huber loss did, however, it failed to turn the lizard blue, demonstrating the Goodhart’s law [10]. Although Huber loss is the best overall loss, all experiments used the baseline MSE loss. Furthermore, these loss functions were selected because of their relation to the color change task. Other loss functions might work better for other experiments such as limb removal.

Other than addressing the stated limitations and issues in the previous paragraphs, there are several directions the future work could follow. Asymmetrical perturbation matrix would allow for a wider search space and different resulting attack strategies. Improving efficiency is also crucial as CMA-ES currently needs roughly three times more NCA forward passes than the gradient-based baseline. Other strategies such as gray-box attacks, natural evolution strategies, or other approaches could reduce the cost. Furthermore, we have not

fully replicated the other experiments by Ranzazzo et al. [3], such as the removal of limbs, combining multiple perturbations at the same time, and even adversarial injections. Doing so would involve a different parameter configuration setting and different fitness function and more. While the black-box model performed well so far, and we believe it would perform well on these task as well, it would be interesting to see how well it compares. Therefore, future work should explore and replicate these other experiments as well and see how the black-box model compares. Finally, future work should test if the perturbation matrix found by CMA-ES would corrupt the defense layers used by Xu et al. [2] inside their Vision Transformers.

Conclusion

Overall, our query-only evolutionary attack comes close to the white-box gradient baseline. It reproduces the same targeted reprogramming and reaches a visually similar result (with near-identical SSIM and comparable LPIPS, 2D-FFT, and TSS). Within the same 30-minute budget the gradient baseline still converges to a somewhat lower loss and slightly sharper detail, but our attack lands in the same range rather than being clearly worse. This closeness comes at a cost, as CMA-ES needs roughly three times more NCA forward passes and more energy to get there. At the same time it shows some interesting properties: it achieves its results with minimal changes to the starter perturbation matrix and appears to fail more gracefully than the gradient approach from a fully random start. The attack also generalises beyond the colour change, approximating a different target (the butterfly) by simply swapping the target image, with the choice of fitness function further shaping the result. We do, however, confirm that the attacks are not permanent, as the NCA regenerates its original form within a few hundred steps once the perturbation is switched off.

We have shown that a white-box attack is not needed to successfully attack NCA’s and that they may not be as robust as they were priorly considered.

References

- [1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, “Growing neural cellular automata,” *Distill*, vol. 5, no. 2, e23, 2020. [Online]. Available: <https://distill.pub/2020/growing-ca> doi: 10.23915/distill.00023.
- [2] Y. Xu, T. Zhang, and S. Süssstrunk, “AdaNCA: Neural cellular automata as adaptors for more robust vision transformer,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 25709–25746, 2024.
- [3] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, “Adversarial reprogramming of neural cellular automata,” *Distill*, vol. 6, no. 5, e00027-004, 2021. [Online]. Available: <https://distill.pub/selforg/2021/adversarial> doi: 10.23915/distill.00027.004.
- [4] L. Cavuoti, F. Sacco, E. Randazzo, and M. Levin, “Adversarial takeover of neural cellular automata,” in *Artificial Life Conference Proceedings 34*, 2022, p. 38. MIT Press.
- [5] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [6] S. Greydanus, “Studying growth with neural cellular automata,” blog post and PyTorch implementation, 2022. [Online]. Available: <https://greydanus.github.io/2022/05/24/studying-growth>
- [7] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018, pp. 586–595.
- [9] P. J. Huber, “Robust Estimation of a Location Parameter,” *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964. [Online]. Available: <https://doi.org/10.1214/aoms/1177703732>
- [10] M. Strathern, “‘Improving ratings’: audit in the British University system,” *European Review*, vol. 5, no. 3, pp. 305–321, 1997. [Online]. Available: <https://doi.org/10.1002/eur.10027>

1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4

Author Contribution

Aaron : Developed the core CMA-ES attack framework, including the perturbation matrix parameterization, initialization strategies, and hyperparameter optimization via Optuna sweeps.

Jason : Designed the evaluation metric suite including the Target-Specific Semantic Score rubric (Appendix D), implemented the SSIM, LPIPS, 2D-FFT, and TSS pipeline used to assess all experiments, set up and ran the gradient-based baseline for comparison, and conducted the environmental impact analysis

David : Implemented and ran the lizard-to-butterfly experiment, the perturbation on/off recovery experiment, and the comparison of different fitness functions.

Tasks were divided roughly equally, with each member independently running their experiments and sharing results with the group. All members contributed to the writing of this report, primarily reporting on their own experiments, with collective involvement in the introduction, discussion, and conclusion.

Code

All code used in this project can be found at <https://github.com/DavidMagaram/NaturalComputingFinal>. All experiments were performed using Jupyter notebooks that were developed in Google Colab using a T4 GPU, and can be located in the *Codee* folder. No additional installation or configuration is needed.

Appendix A: Hyperparameter Optimization using Optuna

We used the Optuna library to find the best hyperparameters for the CMA-ES attack. We divided the search into two consecutive sweeps to manage computational time. The first sweep ran thirty trials over twenty minutes and applied early pruning after five minutes to stop bad runs. Figure 18 shows that this initial search favored a batch size of 4 and a population size of 32 or 64. A higher decay frequency also performed better. Figure 19 confirms that batch size was the most important hyperparameter by a large margin.

For the second sweep, we narrowed the search space. We fixed the batch size to 4 based on the previous results to tune the remaining parameters. This second sweep ran twenty trials over twenty minutes and again used early pruning at five minutes. As shown in Figure 20, this run favored a population size of 64, a higher initial standard deviation, and a higher decay frequency. Figure 21 shows the importance of these remaining parameters.

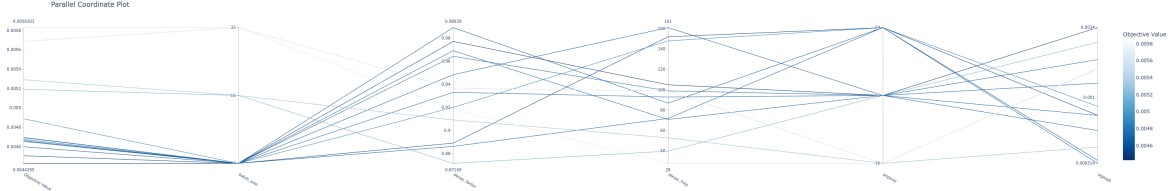


Figure 18: Highlighting the most favorable parameters within the thirty trials of the first Optuna sweep.

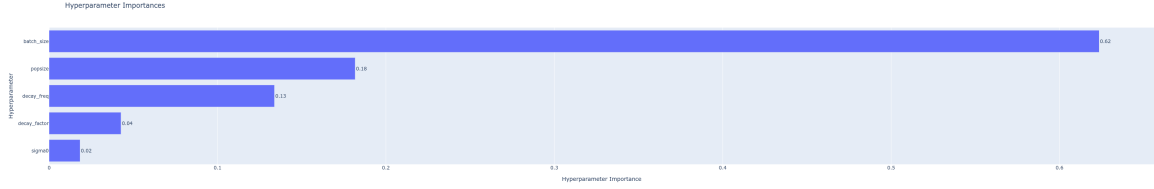


Figure 19: Hyperparameter importance during the first Optuna sweep.

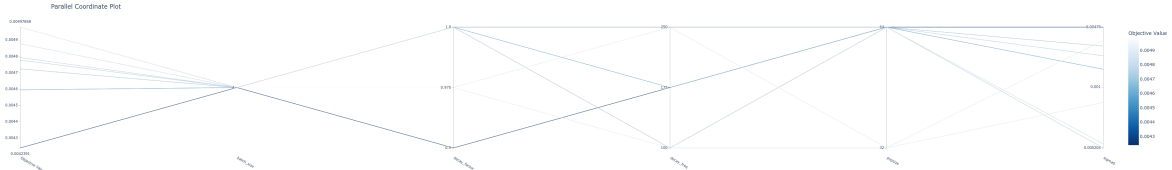


Figure 20: Second optuna sweep with less options as the winners from the first round were reintroduced. This visual highlights the most favorable parameters within the twenty trials.

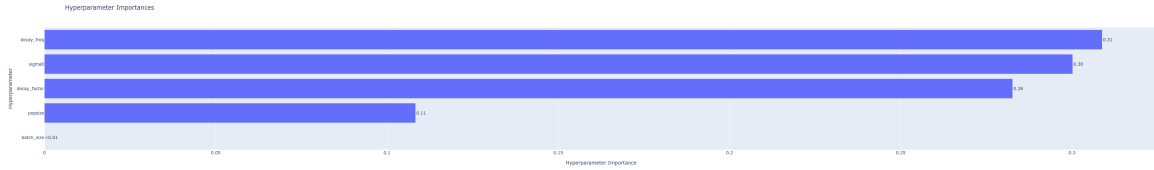


Figure 21: Hyperparameter importance during the second Optuna sweep.

Appendix B: Additional Images for Experiment 4

Target | identity | gradient-start | random (grown from the empty seed)



gradient (Phase 1) growth (steps [0, 20, 40, 60, 80, 100, 120]):



identity growth (steps [0, 20, 40, 60, 80, 100, 120]):



Figure 22: Results from Experiments starting with different matrixes.

gradient-start growth (steps [0, 20, 40, 60, 80, 100, 120]):



random growth (steps [0, 20, 40, 60, 80, 100, 120]):

.

Healthy | gradient | identity | gradient-start | random (mutation applied to a grown lizard)



Figure 23

Appendix C: Additional Images for Experiment 4 Addition

Further graphics plots/graphics from training a gradient-based attack starting on a random starter perturbation matrix:

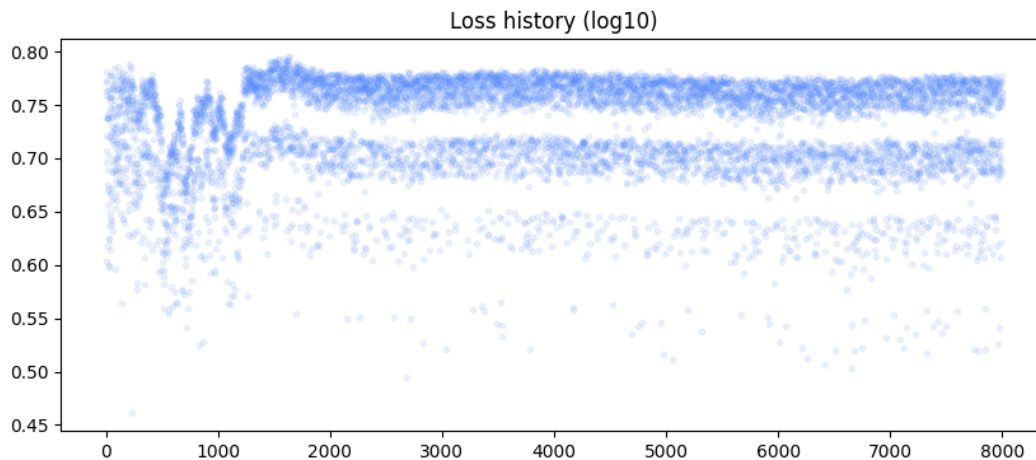


Figure 24: The $\log_{10}$ loss of the gradient-based attack trained for 8000 steps with a random starter perturbation matrix. The y-axis shows a the $\log_{10}$ scale from 0.45 to 0.80.

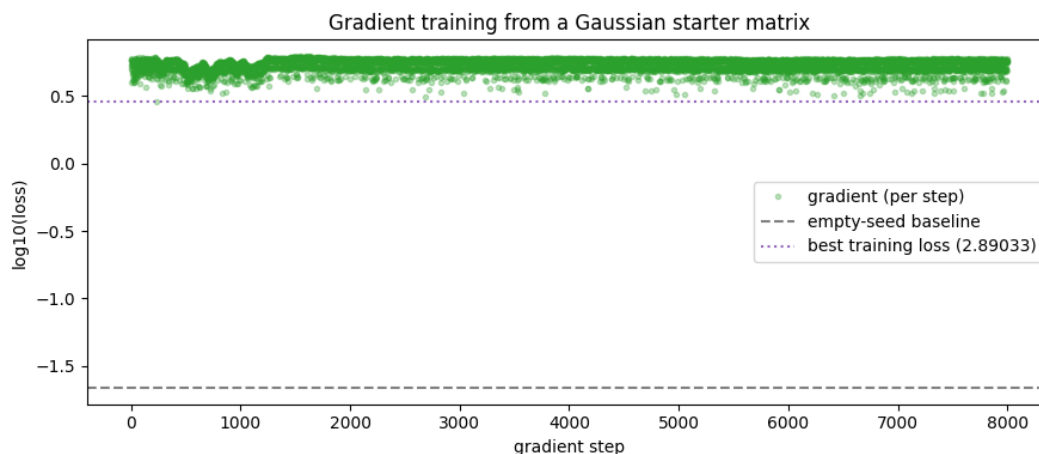


Figure 25: The $\log_{10}$ loss of the gradient-based attack trained for 8000 steps with a random starter perturbation matrix. The empty-seed baseline has been given as a reference point.

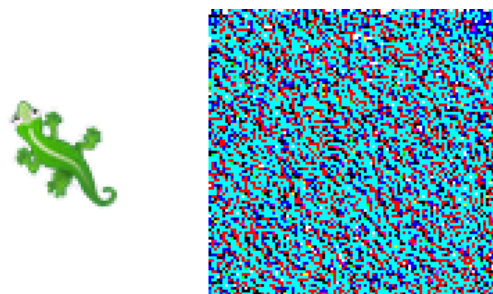


Figure 26: Left: fully grown lizard, original NCA lizard. Right: gradient-based attack applied to an already grown structure.

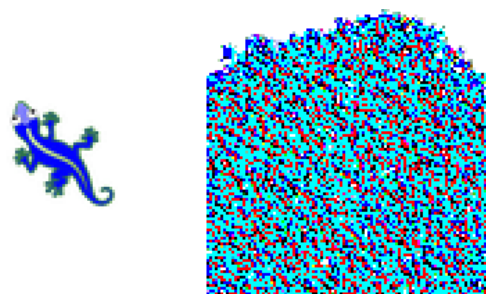


Figure 27: Left: the target lizard. Right: gradient-based attack grown from a single seed pixel.

Appendix D: Rubric for the Target-Specific Semantic Score (TSS)

Feature	1 - Poor (Highly Degraded)	2 - Fair (Significant Artifacts)	3 - Good (Minor Flaws)	4 - Excellent (Target Match)
Granularity	Edges are highly unstable; explosive noise, or completely blurred boundaries.	Significant noise or artifacting around the borders; toes are indistinguishable blobs.	Edges are mostly clean.	Crisp pixelated edges; distinct toes/limbs, with minimal to no stray pixels.
Bodily Parts	Unrecognizable mass; missing a tail or multiple limbs.	Recognizable as a lizard, but severely deformed (e.g., missing one leg, severely stunted/bloated tail).	All four limbs and tail are present; minor structural anomalies or slightly incorrect proportions.	Excellent anatomical structure; four distinct legs with toes, a clearly defined head, and a smoothly curving tail.
Blueness	Deep blue is largely absent, completely overtaken by noise, or shifted to the wrong hue.	Blue is present but patchy, washed out, or heavily mixed with incorrect stray colors.	Body is predominantly the correct deep blue; minor color bleeding or slight noise in the torso.	Solid, cohesive deep blue body that contrasts sharply with the dorsal stripe and the head.
Dorsal Stripe	Stripe is completely missing or unrecognizable.	Stripe is visible but heavily fragmented, scattered, or significantly the wrong color/thickness.	Stripe is visible and mostly continuous down the back, but may have minor interruptions or jaggedness.	Distinct, continuous pale white stripe running cleanly from the base of the head down the spine.
Head Features	Head is indistinguishable from the body; eyes are completely missing.	Head is somewhat distinct, but eyes are blurry, merged into a single blob, or missing pupils.	Head shows lighter coloration; two eyes are present but slightly distorted or asymmetric.	Head is distinctively lighter (light purple/blue); two clear white eyes with distinct black pupils are perfectly placed.

Table 5: Visual Fidelity Rubric for the NCA Lizard Target Image.